

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Харківський національний університет імені В.Н. Каразіна

Навчально-науковий інститут комп'ютерних наук та штучного інтелекту

Кафедра математичного моделювання та аналізу даних

ЛАБОЛАТОРНА РОБОТА № 4

на тему: «Вивчення DML на прикладі створення функцій
для роботи з БД»

Виконав: студент 3 курсу групи кс31
Спеціальності 124 «Комп'ютерні
науки»

Кірієнко Данило

Прийняв: к.т.н., доцент каф. ММ та
АД

Коробчинський К.П.

Мета роботи:

Метою даної роботи є ознайомлення з концепцією функцій у MSSQL, зокрема скалярних та віконних функцій, та їх застосування у практичних запитах. Робота передбачає вивчення можливостей скалярних функцій для обробки та трансформації даних у базах даних

Завдання 1.

Пройти тестування та додати скріншот з оцінкою у звіт.

Завдання 2.

За своїм варіантом (основного, по якому були виконані практичні роботи 3, 4, 5 та 6) додати опис предметної області (скопювати з варіанту), логічну і фізичну моделі (скопювати з практичної роботи №5). Наступні запити виконати в зручному для Вас середовищі: Docker, Віртуальна машина з MSSQL, файлова БД або встановлений на локальному комп'ютері екземпляр MSSQL.

Завдання 3.

Використовуючи скалярний тип функцій зробити приклад використання власної функції. Зробити 3 приклади за тематикою Вашого варіанту.

- Якщо потрібно, використовуйте вбудовані, віконні або агреговані функції.
- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл FUNCTIONS.SQL та оновити його на своїй сторінці на GitHub.

Завдання 4.

Використовуючи Inline тип функцій зробити приклад використання власної функції. Зробити 3 приклади за тематикою Вашого варіанту.

- Якщо потрібно, використовуйте вбудовані, віконні або агреговані функції.
- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл FUNCTIONS.SQL та оновити його на своїй сторінці на GitHub.

Завдання 5.

Використовуючи Multistate тип функцій зробити приклад використання власної функції. Зробити 3 приклади за тематикою Вашого варіанту.

- Якщо потрібно, використайте вбудовані, віконні або агреговані функції.
- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл FUNCTIONS.SQL та оновити його на своїй сторінці на GitHub.

Завдання 6.

Використовуючи існуючі типи виконати запити Вашого варіанта. За тематикою Вашого варіанту є запити, їх потрібно зробити у функціях, звертаю увагу використовувати потрібно відповідний тип функції.

- Якщо потрібно, використайте вбудовані, віконні або агреговані функції.
- До звіту додати скріншоти виконання запитів та їхніх результатів.
- Зберегти код запитів у файл FUNCTIONS.SQL та оновити його на своїй сторінці на GitHub.

Завдання 7. Аналіз продуктивності запитів

Для аналізу продуктивності запитів у MS SQL використайте EXPLAIN (Execution Plan). Додайте скріншоти з поясненням та аналізом попередніх запитів.

Завдання 8.

Завантажити файли до Git-репозиторію та додати посилання у звіт. Репозиторій повинен бути закритим, а доступ відкритий лише для викладачів практики, лабораторних робіт та лектора

(k.korobchinskiy@karazin.ua).

Використовуючи мову T-SQL зробити скрипти.

- створення таблиць та змінення структури таблиць БД – SETUP.SQL;
- занесення інформації в таблиці БД – INSERT.SQL.

- зміни/оновлення/видалення інформації у таблицях БД – UPDATE.SQL;

- запитів – QUERY.SQL.

- функцій – FUNCTIONS.SQL.

Всі фали за архівувати та прикипіти до STM.

Завдання 9.

Виконати запити за Вашим варіантом відповідно до опису предметної області основного варіанта.

Завдання 10.

Додати висновки. До оформити звіт та завантажити на STM.

Хід роботи:

Завдання 2.

Варіант №15

Ферма

На ферме маєтись декілька ділянок землі, їм присвоєні унікальні номери. Кожний ділянку характеризується площею, наявністю або відсутністю зрошення, посіяної в поточному сезоні культурою. Відомі середня урожайність кожної з вирощуваних культур, а також перелік внесених на кожний ділянку в данному сезоні добрив. Для кожної культури відомо: назва, урожайність по ділянках, потрібно ли для неї зрошення, які потрібні добрива. Кожний ділянку обслуговує одна бригада, но будь-яка бригада може обслуговувати більше одного ділянки. Бригада має унікальний номер і характеризується: Ф.І.О. бригадира, Ф.І.О. працівників, кількістю одиниць кожного виду сільськогосподарської техніки. О кожному працівнику відомо, якими видами сільськогосподарських машин (одним, декількома, ні одним) он може керувати. Працівники працюють на кожному з ділянок бригади, користуються технікою, яка є в бригаді.

Запити

- ✓ Визначити площу, яка відведена на фермі під кожну культуру з вказівкою, на якій кількості ділянок посіяна кожна культура;
- ✓ Вивести всіх працівників заданої бригади з переліком сільськогосподарської техніки, на якій вони можуть працювати, якщо така техніка в бригаді маєтись, а також визначити те види техніки, на якій можуть працювати члени бригади, но в бригаді такої техніки немає, хоча вона є в інших бригадах
- д л я ділянок данної бригади знайти культуру з максимальною урожайністю і перевірити, посіяна ли ця культура в поточному сезоні на тому ділянці, де її урожайність максимальна
- ✓ Знайти бригади, які мають заданим видом техніки в кількості, більшою ніж середнє кількість цієї техніки по бригадам на всій фермі;
- ✓ Скласти список ділянок (з іменами їх бригадирів), на яких внесені добрива, не відповідні культурі, або не виконуються вимоги по зрошенню культури.

Транзакції

- ✓ Купити нову сільськогосподарську техніку
- ✓ Найняти/увільнити сільськогосподарських (в тому числі і бригадирів).

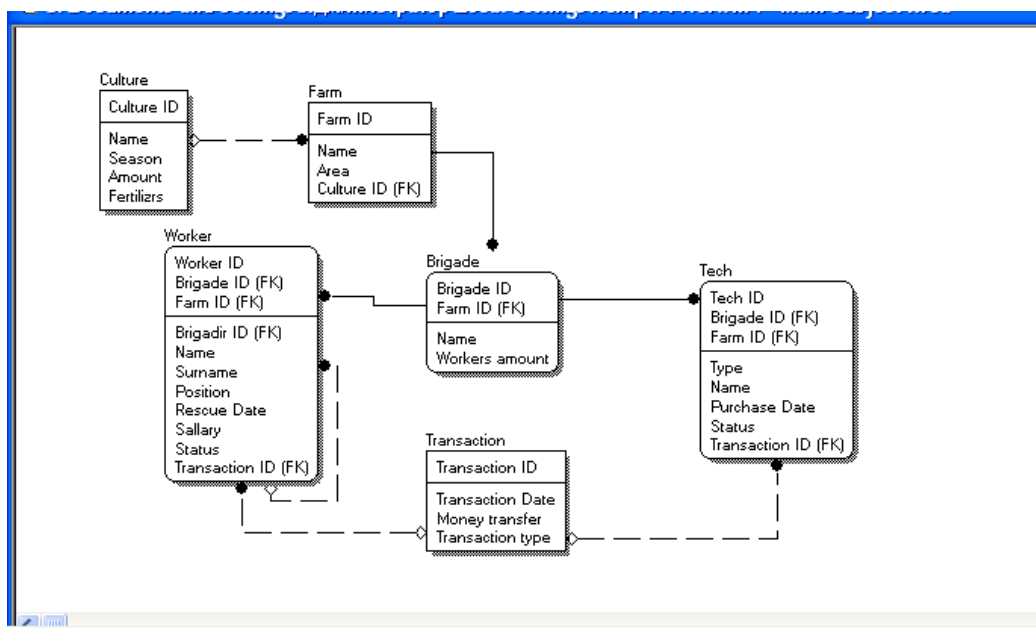


Рисунок 1 - Логічна модель

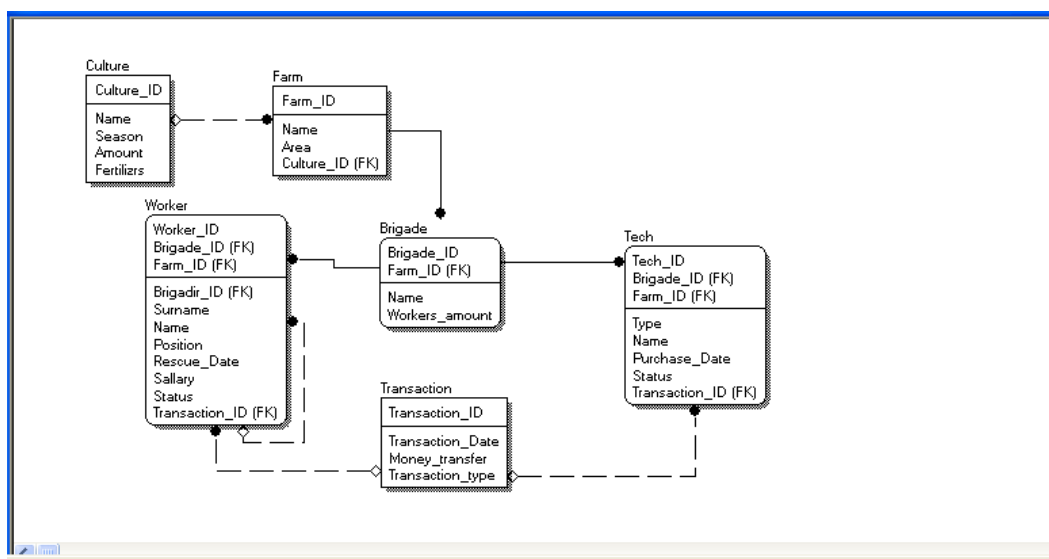


Рисунок 2 - Фізична модель

Завдання 3.

Давайте створемо скалярні функції:

```
CREATE FUNCTION dbo.GetFullName
(
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50)
)
RETURNS NVARCHAR(100)
AS
BEGIN
    RETURN @LastName + ' ' + @FirstName
```

```

END
GO

CREATE FUNCTION dbo.GetSalaryLevel (@Salary DECIMAL(10,2))
RETURNS NVARCHAR(20)
AS
BEGIN
    DECLARE @Level NVARCHAR(20)
    IF @Salary >= 15000
        SET @Level = 'High'
    ELSE IF @Salary >= 10000
        SET @Level = 'Medium'
    ELSE
        SET @Level = 'Low'
    RETURN @Level
END
GO

CREATE FUNCTION dbo.GetTechStatusLabel (@Status NVARCHAR(50))
RETURNS NVARCHAR(50)
AS
BEGIN
    DECLARE @Label NVARCHAR(50)
    IF @Status = 'Working'
        SET @Label = 'Operational'
    ELSE IF @Status = 'Broken'
        SET @Label = 'Not Operational'
    ELSE IF @Status = 'UnderRepair'
        SET @Label = 'Maintenance'
    ELSE
        SET @Label = 'Unknown'
    RETURN @Label
END
GO

```

Тепер, виконуючи відповідні користувацьки запити ми будемо отримувати результат. Має бути так:

1. SELECT dbo.GetFullName(Name, Surname) AS FullName FROM Worker;
Цей запит дасть повне ім'я
2. SELECT Name, Surname, Salary, dbo.GetSalaryLevel(Salary) AS SalaryLevel FROM Worker;
Повертає значення відповідне зарплатні

3. SELECT Name, Status, dbo.GetTechStatusLabel(Status) AS StatusLabel
FROM Tech;

Дає «спрощений» статус техніки.

Перевіримо, виконаємо ці запити:

Results		Messages	
FullName			
1	Sumame1 Name1		
2	Sumame2 Name2		
3	Sumame3 Name3		
4	Sumame4 Name4		
5	Sumame5 Name5		
6	Sumame6 Name6		
7	Sumame7 Name7		
8	Sumame8 Name8		
Name	Sumame	Salary	SalaryLevel
1	Name1	Sumame1	12542.85 Medium
2	Name2	Sumame2	7488.66 Low
3	Name3	Sumame3	5416.00 Low
4	Name4	Sumame4	13896.34 Medium
5	Name5	Sumame5	17069.79 High
6	Name6	Sumame6	13497.87 Medium
7	Name7	Sumame7	11923.01 Medium
8	Name8	Sumame8	5227.77 Low
Name	Status	StatusLabel	
1	Combine	Broken	Not Operational
2	Plow	UnderRepair	Maintenance
3	Tractor	Working	Operational
4	Combine	Broken	Not Operational
5	Seeder	Broken	Not Operational
6	Plow	Broken	Not Operational
7	Seeder	Broken	Not Operational
8	Plow	Broken	Not Operational

Рисунок 3 - результат скалярних функцій

Завдання 4.

Тепер створення inline функцій:

```
CREATE FUNCTION dbo.fn_WorkersByStatus (@Status NVARCHAR(50))
RETURNS TABLE
AS
RETURN
    SELECT Worker_ID, Name, Surname, Position, Salary
    FROM Worker
    WHERE Status = @Status;
GO

CREATE FUNCTION dbo.fn_TechDetailsByState (@State NVARCHAR(50))
RETURNS TABLE
AS
RETURN
    SELECT
        Tech_ID,
        Name,
        Type,
        Status,
        Purchase_Date,
        DATEDIFF(YEAR, Purchase_Date, GETDATE()) AS AgeInYears
    FROM Tech
    WHERE Status = @State
    ORDER BY Purchase_Date DESC;
GO

CREATE FUNCTION dbo.fn_HighSalaryWorkers()
RETURNS TABLE
AS
RETURN
    SELECT Worker_ID, Name, Surname, Salary
    FROM Worker
    WHERE Salary > (SELECT AVG(Salary) FROM Worker);
GO
```

Маємо такі запити:

1. SELECT * FROM dbo.fn_WorkersByStatus('Active');
Працює так само як селект, це скоріше для розуміння як працюють inline
2. SELECT * FROM dbo.fn_TechDetailsByState('Broken');
Рахує скільки часу техніці з статусом broken

3. `SELECT * FROM dbo.fn_HighSalaryWorkers();`

Показує в кого зарплатня вища за середню

Спробуємо їх:

	Worker_ID	Name	Surname	Position	Salary	
1	1	Name1	Sumame1	Worker	12542.85	
2	2	Name2	Sumame2	Worker	7488.66	
3	5	Name5	Sumame5	Foreman	17069.79	
4	6	Name6	Sumame6	Worker	13497.87	
5	10	Name10	Sumame10	Tractor driver	19277.26	
6	12	Name12	Sumame12	Tractor driver	16111.29	
7	16	Name16	Sumame16	Worker	12319.07	
8	20	Name20	Sumame20	Tractor driver	6927.15	

	Tech_ID	Name	Type	Status	Purchase_Date	AgeInYears
1	1	Combine	Type 4	Broken	2023-12-17	2
2	4	Combine	Type 5	Broken	2022-09-14	3
3	5	Seeder	Type 2	Broken	2022-08-14	3
4	6	Plow	Type 4	Broken	2023-07-05	2
5	7	Seeder	Type 1	Broken	2023-12-17	2
6	8	Plow	Type 5	Broken	2022-07-10	3
7	11	Combine	Type 3	Broken	2024-12-18	1
8	12	Plow	Type 1	Broken	2023-04-29	2

	Worker_ID	Name	Surname	Salary
1	1	Name1	Sumame1	12542.85
2	4	Name4	Sumame4	13896.34
3	5	Name5	Sumame5	17069.79
4	6	Name6	Sumame6	13497.87
5	10	Nam...	Sumam...	19277.26
6	12	Nam...	Sumam...	16111.29
7	14	Nam...	Sumam...	17799.32
8	16	Nam...	Sumam...	12319.07

Рисунок 4 - скріншот інлайн

Завдання 5.

Тепер, давайте спробуємо Multistate. Тут є сенс спробувати більш комплексні функції:

```
CREATE FUNCTION dbo.fn_WorkersWithBonus ()
RETURNS @Result TABLE (
    Worker_ID INT,
    Name NVARCHAR(50),
    Surname NVARCHAR(50),
    Salary DECIMAL(10,2),
    Bonus DECIMAL(10,2)
)
```

```

AS
BEGIN
    INSERT INTO @Result
    SELECT
        Worker_ID, Name, Surname, Salary,
        Salary * 0.10 AS Bonus
    FROM Worker
    WHERE Salary * 0.10 > 1500;

    RETURN
END
GO
CREATE FUNCTION dbo.fn_TechAgeFilter(@MinYears INT)
RETURNS @Result TABLE (
    Tech_ID INT,
    Name NVARCHAR(50),
    Purchase_Date DATE,
    Age INT
)
AS
BEGIN
    INSERT INTO @Result
    SELECT
        Tech_ID,
        Name,
        Purchase_Date,
        DATEDIFF(YEAR, Purchase_Date, GETDATE()) AS Age
    FROM Tech
    WHERE DATEDIFF(YEAR, Purchase_Date, GETDATE()) >= @MinYears;

    RETURN
END
GO
CREATE FUNCTION dbo.fn_TransactionsOverThreshold(@MinAmount DECIMAL(10,2))
RETURNS @Result TABLE (
    Transaction_ID INT,
    Transaction_type NVARCHAR(50),
    Money_transfer DECIMAL(10,2)
)
AS
BEGIN
    INSERT INTO @Result
    SELECT
        Transaction_ID,
        Transaction_type,
        Money_transfer
    FROM [Transaction]
    WHERE Money_transfer > @MinAmount;

    RETURN
END
GO

```

Знову три функції:

1. `SELECT * FROM dbo.fn_WorkersWithBonus();`
Рахує премію 10% від зарплатні і виводить лише тих, в кого премія більше 1500 грн.
2. `SELECT * FROM dbo.fn_TechAgeFilter(2);`
Виводить техніку якій не менше 2 років
3. `SELECT * FROM dbo.fn_TransactionsOverThreshold(30000);`
Виводить транзакції сума яких менше 30к

Results

Messages

	Worker_ID	Name	Surname	Salary	Bonus
1	5	Name5	Surname5	17069.79	1706.98
2	10	Name10	Surname10	19277.26	1927.73
3	12	Name12	Surname12	16111.29	1611.13
4	14	Name14	Surname14	17799.32	1779.93
5	24	Name24	Surname24	15834.21	1583.42
6	26	Name26	Surname26	17302.16	1730.22
7	28	Name28	Surname28	15896.35	1589.64
8	32	Name32	Surname32	19581.16	1958.12

	Tech_ID	Name	Purchase_Date	Age
1	1	Combine	2023-12-17	2
2	4	Combine	2022-09-14	3
3	5	Seeder	2022-08-14	3
4	6	Plow	2023-07-05	2
5	7	Seeder	2023-12-17	2
6	8	Plow	2022-07-10	3
7	10	Plow	2022-12-05	3
8	12	Plow	2023-04-29	2

	Transaction_ID	Transaction_type	Money_transfer
1	2	Salary	39883.07
2	3	Salary	35691.88
3	4	Salary	30451.71
4	5	Sale	43028.27
5	7	Salary	43543.71
6	9	Sale	33430.16
7	10	Repair	35122.27
8	18	Purchase	48321.86

Рисунок 5 - Multistate виклики

Завдання 6.

По завданню маємо 4 запити (один комплексний):

1. Визначити площу, яка відведена на фермі під кожну культуру з вказанням, на якій кількості ділянок висаджена кожна культура;
2. Вивести всіх працівників заданої бригади з переліком сільськогосподарської техніки, на якій вони можуть працювати, якщо така техніка в бригаді є, а також визначити ті види техніки, на якій можуть працювати члени бригади, але в бригаді такої техніки немає, хоча вона є в інших бригадах
- для ділянок даної бригади знайти культуру з максимальною врожайністю і перевірити, висаджена чи ця культура в поточному сезоні на тому ділянці, де її врожайність максимальна
3. Знайти бригади, які мають заданим видом техніки в кількості більше середньої кількості цієї техніки по бригадах на всій фермі:
4. Скласти список ділянок (з іменами їх керівників), на яких внесені добрива, не відповідні культурі, або не виконуються вимоги по зрошенню культури.

Переходимо до реалізації:

1. Інлайн

```
CREATE FUNCTION dbo.fn_CultureAreaStats ()
RETURNS TABLE
AS
RETURN
    SELECT
        c.Name AS CultureName,
        c.Season,
        COUNT(f.Farm_ID) AS FarmsCount,
        SUM(f.Area) AS TotalArea
    FROM Culture c
    JOIN Farm f ON c.Culture_ID = f.Culture_ID
    GROUP BY c.Name, c.Season;
GO
```

Робимо запит: `SELECT * FROM dbo.fn_CultureAreaStats();`

	CultureName	Season	FarmsCount	TotalArea
1	Barley	Autumn	8	1097
2	Corn	Autumn	9	1128
3	Potato	Autumn	11	1400
4	Sunflower	Autumn	7	877
5	Wheat	Autumn	3	511
6	Barley	Spring	13	1699
7	Corn	Spring	6	822
8	Potato	Spring	1	143
9	Sunflower	Spring	3	384
10	Wheat	Spring	3	491
11	Barley	Sum...	4	522
12	Corn	Sum...	5	415
13	Potato	Sum...	7	902
14	Sunflower	Sum...	11	1565
15	Wheat	Sum...	9	1300

Рисунок 6 - результат виконання запису 1

2. Тут виникла невеличка проблема. У запиті сказано, що треба знати перелік техніки, якою володіють люди з бригади, але наразі моя система не має такої можливості. Тому, в файл UPDATE додаємо розширення:

```

ALTER TABLE Worker
ADD Tech_Type NVARCHAR(50) NULL;

WITH Types AS (
    SELECT DISTINCT Type FROM Tech
),
Randomized AS (
    SELECT
        w.Worker_ID,
        t.Type,
        ROW_NUMBER() OVER (ORDER BY NEWID()) AS rn
    FROM Worker w
    CROSS JOIN Types t
)

UPDATE w
SET Tech_Type = r.Type
FROM Worker w
JOIN (
    SELECT Worker_ID, Type,
        ROW_NUMBER() OVER (PARTITION BY Worker_ID ORDER BY NEWID()) AS
rnk
    FROM Randomized
) r ON w.Worker_ID = r.Worker_ID
WHERE r.rnk = 1 AND w.Worker_ID <= 80;

```

Таким чином, ми додали Type з типами техніки у worker, і потім рандомно дали їх першим 80 працівників ферми. Тепер worker і tech мають тип техніки. Також додамо ще людей, щоб вийшло що в бригаді декілька людей. Далі реалізуємо запит 2, тип Multistatement:

```
CREATE FUNCTION dbo.fn_BrigadeTechSkills
(
    @BrigadeID INT,
    @FarmID INT
)
RETURNS @Result TABLE (
    WorkerName NVARCHAR(100),
    TechType NVARCHAR(50),
    TechAvailableInBrigade BIT,
    TechExistsInOtherBrigades BIT
)
AS
BEGIN
    INSERT INTO @Result
    SELECT
        w.Name + ' ' + w.Surname AS WorkerName,
        w.Tech_Type,
        CASE
            WHEN EXISTS (
                SELECT 1 FROM Tech t
                WHERE t.Brigade_ID = @BrigadeID
                    AND t.Farm_ID = @FarmID
                    AND t.Type = w.Tech_Type
            ) THEN 1 ELSE 0
        END AS TechAvailableInBrigade,
        CASE
            WHEN EXISTS (
                SELECT 1 FROM Tech t
                WHERE NOT (t.Brigade_ID = @BrigadeID AND t.Farm_ID =
@FarmID)
                    AND t.Type = w.Tech_Type
            ) THEN 1 ELSE 0
        END AS TechExistsInOtherBrigades
    FROM Worker w
    WHERE w.Brigade_ID = @BrigadeID
        AND w.Farm_ID = @FarmID
        AND w.Tech_Type IS NOT NULL;

    RETURN;
END
GO
```

Після виконуємо SELECT * FROM dbo.fn_BrigadeTechSkills(1, 1);:

	WorkerName	TechType	TechAvailableInBrigade	TechExistsInOtherBrigades
1	Name1 Surname1	Type 2	0	1
2	John Doe	Type 4	1	1
3	Alice Smith	Type 1	0	1

Рисунок 7 - запит `SELECT * FROM dbo.fn_BrigadeTechSkills(1, 1);`

Далі друга частина. Для визначення урожайності беремо amount/area. Тип Multistatement:

```
CREATE FUNCTION dbo.fn_BrigadeMaxYieldCulture
(
    @FarmID INT,
    @Season NVARCHAR(20)
)
RETURNS @Result TABLE (
    CultureName NVARCHAR(50),
    Yield FLOAT,
    Area INT,
    Amount INT,
    IsPlantedNow BIT
)
AS
BEGIN
    DECLARE @MaxYieldCultureID INT

    SELECT TOP 1 @MaxYieldCultureID = c.Culture_ID
    FROM Farm f
    JOIN Culture c ON f.Culture_ID = c.Culture_ID
    WHERE f.Farm_ID = @FarmID
    ORDER BY CAST(c.Amount AS FLOAT) / NULLIF(f.Area, 0) DESC;

    INSERT INTO @Result
    SELECT
        c.Name,
        CAST(c.Amount AS FLOAT) / NULLIF(f.Area, 0) AS Yield,
        f.Area,
        c.Amount,
        CASE WHEN c.Season = @Season THEN 1 ELSE 0 END AS IsPlantedNow
    FROM Farm f
    JOIN Culture c ON f.Culture_ID = c.Culture_ID
    WHERE f.Farm_ID = @FarmID AND c.Culture_ID = @MaxYieldCultureID;

    RETURN;
END
GO
```

Після виконуємо `SELECT * FROM dbo.fn_BrigadeMaxYieldCulture(1, 'Summer');`:

Results		Messages			
	CultureName	Yield	Area	Amount	IsPlantedNow
1	Barley	0,432835820895522	67	29	1

Рисунок 8 - виконання `SELECT * FROM dbo.fn_BrigadeMaxYieldCulture(1, 'Summer');`

Для розуміння вихідної таблиці – друге поле це урожайність, тобто навища у барлі з 0,4. В 5 полі 1 означає, що культура вже посаджена в цьому сезоні.

3. Тут полегше, тому тип інлайн:

```
CREATE FUNCTION dbo.fn_BrigadesWithAboveAvgTech (@Type NVARCHAR(50))
RETURNS TABLE
AS
RETURN
WITH TechPerBrigade AS (
    SELECT Brigade_ID, Farm_ID, COUNT(*) AS TechCount
    FROM Tech
    WHERE Type = @Type
    GROUP BY Brigade_ID, Farm_ID
),
AvgCount AS (
    SELECT AVG(CAST(TechCount AS FLOAT)) AS AvgTech FROM TechPerBrigade
)
SELECT tpb.*
FROM TechPerBrigade tpb
CROSS JOIN AvgCount ac
WHERE tpb.TechCount > ac.AvgTech;
GO
```

Робимо запит `SELECT * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');`:

Results		Messages	
Brigade_ID	Farm_ID	TechCount	

Рисунок 9 - `SELECT * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');` (1)

Очевидно результат пустий, оскільки жодна з бригад не перевищує середнє. Давайте спробуємо дати бригаді 52 трошки техніки типу 2, тоді:

Results		Messages	
Brigade_ID	Farm_ID	TechCount	
1	52	2	

Рисунок 10 - `SELECT * FROM dbo.fn_BrigadesWithAboveAvgTech('Type 2');` (2)

Як і очікувалося, тепер ми маємо результат у вигляді 52 бригади.

4. Тут потрібно багато таблиць та прописати правила для добрива, тому буде зручніше використати тип Multistatement:

```

CREATE FUNCTION dbo.fn_FarmFertilizerCheck()
RETURNS @Result TABLE (
    Farm_ID INT,
    Culture_Name NVARCHAR(50),
    Area INT,
    Fertilizers NVARCHAR(50),
    BrigadierSurname NVARCHAR(50),
    Warning NVARCHAR(100)
)
AS
BEGIN
    INSERT INTO @Result
    SELECT
        f.Farm_ID,
        c.Name,
        f.Area,
        c.Fertilizers,
        bdr.Surname,
        'Fertilizer does not match culture' AS Warning
    FROM Farm f
    JOIN Culture c ON f.Culture_ID = c.Culture_ID
    JOIN Brigade b ON b.Farm_ID = f.Farm_ID
    JOIN Worker bdr ON bdr.Worker_ID = b.Brigade_ID
                     AND bdr.Brigade_ID = b.Brigade_ID
                     AND bdr.Farm_ID = b.Farm_ID
    WHERE
        (c.Name = 'Wheat' AND c.Fertilizers <> 'Nitrogen') OR
        (c.Name = 'Barley' AND c.Fertilizers <> 'Organic') OR
        (c.Name = 'Corn' AND c.Fertilizers <> 'Complex') OR
        (c.Name = 'Sunflower' AND c.Fertilizers <> 'Potassium') OR
        (c.Name = 'Potato' AND c.Fertilizers <> 'Nitrogen');

    RETURN;
END
GO

```

Виконаємо запит `SELECT * FROM dbo.fn_FarmFertilizerCheck();`:

	Farm_ID	Culture_Name	Area	Fertilizers	BrigadierSumame	Waming
1	2	Com	84	Nitrogen	Sumame2	Fertilizer does not match culture
2	3	Wheat	137	Potassium	Sumame3	Fertilizer does not match culture
3	4	Wheat	140	Organic	Sumame4	Fertilizer does not match culture
4	5	Potato	96	Complex	Sumame5	Fertilizer does not match culture
5	6	Sunflower	105	Phosphorus	Sumame6	Fertilizer does not match culture
6	7	Barley	71	Nitrogen	Sumame7	Fertilizer does not match culture
7	9	Barley	106	Potassium	Sumame9	Fertilizer does not match culture
8	10	Sunflower	63	Complex	Sumame10	Fertilizer does not match culture
9	11	Wheat	192	Potassium	Sumame11	Fertilizer does not match culture
10	12	Sunflower	127	Complex	Sumame12	Fertilizer does not match culture
11	13	Sunflower	195	Organic	Sumame13	Fertilizer does not match culture
12	14	Com	62	Potassium	Sumame14	Fertilizer does not match culture
13	15	Wheat	188	Complex	Sumame15	Fertilizer does not match culture
14	16	Potato	143	Complex	Sumame16	Fertilizer does not match culture
15	17	Sunflower	169	Complex	Sumame17	Fertilizer does not match culture
16	18	Wheat	111	Complex	Sumame18	Fertilizer does not match culture
17	19	Com	145	Potassium	Sumame19	Fertilizer does not match culture
18	20	Sunflower	190	Organic	Sumame20	Fertilizer does not match culture
19	22	Sunflower	106	Complex	Sumame22	Fertilizer does not match culture
20	23	Barley	180	Complex	Sumame23	Fertilizer does not match culture
21	24	Sunflower	150	Nitrogen	Sumame24	Fertilizer does not match culture
22	25	Potato	187	Complex	Sumame25	Fertilizer does not match culture
23	26	Barley	173	Potassium	Sumame26	Fertilizer does not match culture
24	27	Barley	160	Complex	Sumame27	Fertilizer does not match culture

Рисунок 11 - запит `SELECT * FROM dbo.fn_FarmFertilizerCheck();`

Результатів у вихідній таблиці 87, тож все ж таки 13 бригадирів виявилися достатньо кваліфікованими.

Завдання 6.

Проаналізуємо наші запити:

Query 1: Query cost (relative to the batch): 47%
`SELECT * FROM dbo.fn_CultureAreaStats()`

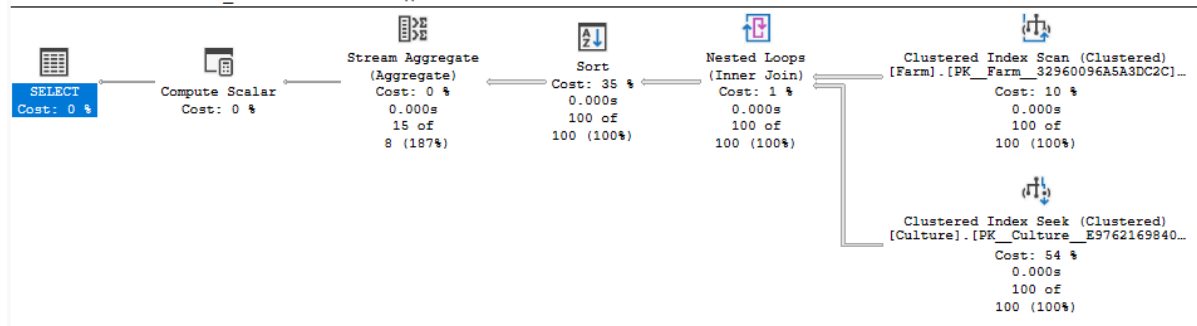


Рисунок 12 - аналіз запиту 1

Найдорожка операція Seek по Culture → 54%, бо ймовірно дуже багато звернень по ключу. Sort має 35% вартості, бо виконується для GROUP BY:

- Clustered Index Seek (Culture) - Швидко знаходить рядки в Culture по PK(54%)
- Clustered Index Scan (Farm) - Читає всі рядки з Farm, бо фільтру нема (10%)
- Nested Loops - Зв'язує Farm з Culture(1%)
- Sort - Сортує для групування (GROUP BY)(35%)
- Stream Aggregate - Рахує SUM(Area) + COUNT(*) (0%)

Query 2: Query cost (relative to the batch): 4%
 SELECT * FROM dbo.fn_BrigadeTechSkills(1, 1)

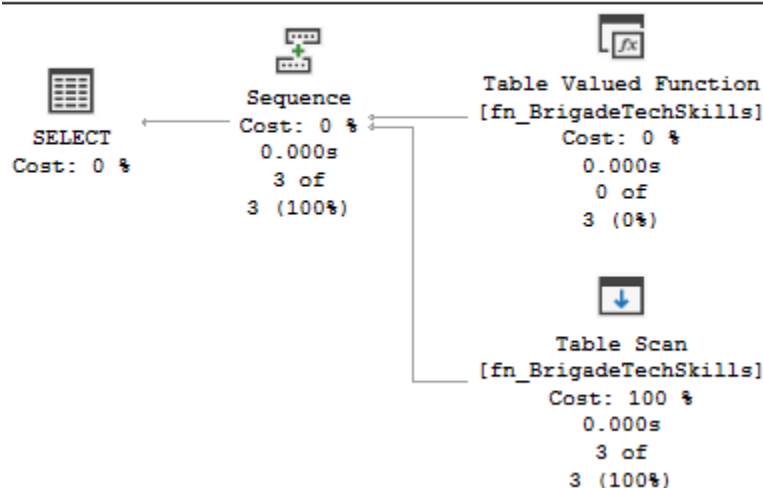


Рисунок 13 - аналіз 2

- Table Valued Function - Це виклик вашої multistatement функції fn_BrigadeTechSkills
- Table Scan - Всі дані, які функція поміщає у @Result, просто скануються без індексу
- Sequence - Обробка послідовності етапів (логіка виконання)

Аналіз не може повністю проаналізувати Multistatement, тому показує все як Table Scan.

Query 3: Query cost (relative to the batch): 4%

```
SELECT * FROM dbo.fn_BrigadeMaxYieldCulture(1, 'Summer')
```

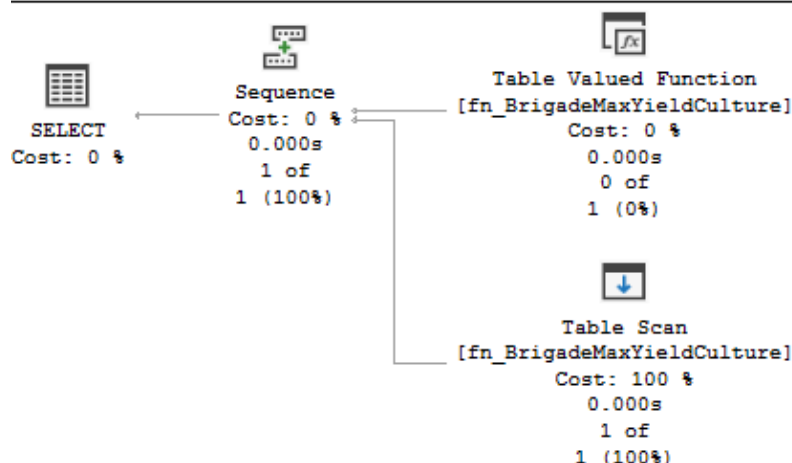


Рисунок 14 - аналіз 3

- Table Valued Function - Функція виконується як чорна коробка
- Table Scan - Сканується вся @Result таблиця всередині функції
- Sequence - Лінійна послідовність кроків

Така сама «проблема» як і з минулом аналізом

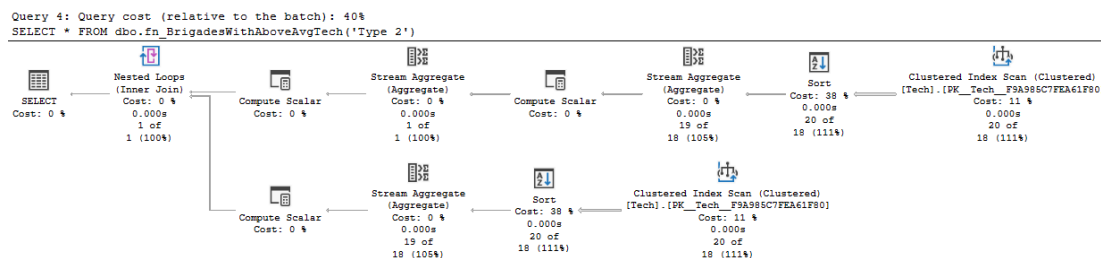


Рисунок 15 - аналіз 4

- Clustered Index Scan (Tech) - Двічі читає всю таблицю Tech, фільтруючи Type = 'Type 2'
- Sort - Сортує дані для GROUP BY Brigade_ID, Farm_ID
- Stream Aggregate - Рахує COUNT(*) і AVG(...)
- Nested Loops - Об'єднує результати — основна логіка JOIN між підзапитами

Те, що тут нема таких проблем як з минулими аналізами ще раз підтверджує, що це інлайн.

Query 5: Query cost (relative to the batch): 5%
 SELECT * FROM dbo.fn_FarmFertilizerCheck()

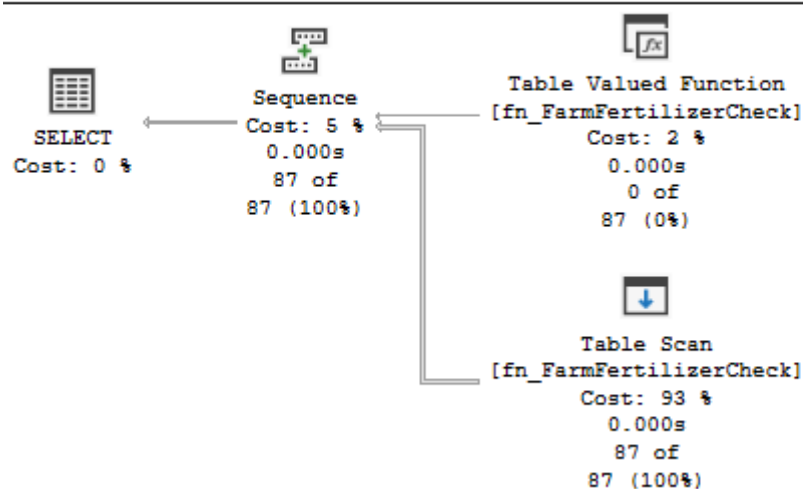


Рисунок 16 - аналіз 5

Не бачу сенсу втретє дублювати те і саме, аналіз не дає нам тут ніякої цікавої інформації.

Висновок:

У ході виконання лабораторної роботи було створено та протестовано різні типи користувацьких функцій у базі даних за тематикою фермерського господарства. Реалізовано скалярні, inline та multistatement табличні функції, які покривають ключові запити варіанта. Було проведено аналіз планів виконання запитів, що дозволило порівняти поведінку функцій різних типів у процесі оптимізації. Inline-функції показали високу ефективність завдяки вбудовуванню в запит, тоді як multistatement-функції забезпечили гнучкість логіки при складніших перевірках.

GitHub з файлами практичної: <https://github.com/ddanq4/3-obd/tree/main/2%20semestr/lab4>